Improving Single-User EMG-to-Text Decoding via Architecture and Training Design

Jingze Fu
UID: 906202950
fujingze1019@g.ucla.edu

Mu Li
UID: 306780665
lim29@ucla.edu

Maoqi Xu
UID: 006770427
mqxu@ucla.edu

Kaiwen Zhao
UID: 606763706
kaiwenzhao2@g.ucla.edu

Abstract

We study the `emg2qwerty` single-user decoding task and analyze which design choices most reduce character error rate (CER) (6). Our group explores several complementary directions, including temporal encoder design, bilateral fusion strategies, and optimization or decoding changes. Across our experiments, changes that strengthen temporal modeling are associated with better performance than the provided TDSConv baseline, although some gains may also depend on optimization and decoding choices. In Branch A, the strongest result in our setup comes from a CNN+GRU hybrid encoder, while Branch B and Branch C suggest that both dilated temporal convolutions and late bilateral fusion can also yield meaningful gains. This report presents the shared setup, the results from each branch, and the main takeaways from comparing them.Code for the project is available in our GitHub repository.[1]

1 Introduction

Silent EMG typing is a sequence decoding problem in which multichannel EMG signals are mapped to character sequences. For this project, rather than restating the general background of sEMG decoding, we focus on the more specific question of which modeling and training choices are most important for reducing CER in the provided single-user benchmark.

Our group approaches this question from several complementary angles. The first branch studies encoder architecture and training design, asking whether recurrent encoders provide a better inductive bias than the provided TDSConv baseline, and whether CNN+RNN hybrids or optimizer/augmentation changes further improve performance. Additional branches investigate alternative temporal architectures and bilateral fusion strategies, allowing us to compare complementary modeling ideas within one report. This organization lets us compare multiple hypotheses within one coherent report instead of presenting disconnected experiments.

The main contributions of this report are:

- A unified comparison of multiple experiment branches on the same single-user EMG-to-text benchmark.
- An empirical study of temporal encoder design, showing strong gains from replacing the baseline encoder with more effective temporal modeling strategies.
- A cross-branch summary of both positive and negative results, including cases where stronger regularization or more complex designs do not help.

[1] `https://github.com/KaiwenZhao123/C247A_Final_Project`

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Paper roadmap. Section 2 describes the shared pipeline and the three experiment branches. Section 3 reports quantitative results and the main observations from each branch. Section 4 summarizes the key insights and limitations.

2 Methods

2.1 Task and evaluation

We follow the provided single-user `emg2qwerty` setting and report character error rate (CER) as the primary metric (6). Unless otherwise stated, all team members use the provided train/validation/test split and compare against the same baseline setup so that results across branches remain directly comparable.

2.2 Shared preprocessing and training pipeline

All experiment branches build on the same overall pipeline: raw EMG signals are converted into input features, passed through an encoder-decoder architecture trained with a CTC objective (1). Depending on the experiment, decoding uses either the default greedy CTC decoder or CTC beam decoding with an external character LM. CTC beam decoding is used throughout Branch A and for the best-performing model in Branch C.

The shared pipeline includes the following components:

- tensor conversion for left/right EMG inputs,
- log-spectrogram feature extraction,
- optional band rotation augmentation,
- optional temporal alignment jitter,
- optional SpecAugment-style masking (2),
- greedy or CTC beam decoding with an external character-level LM.

2.3 Baseline

We use the provided TDSConv-based CTC model as the main reference point (5). This baseline was trained for 150 epochs using the default hyperparameter configuration provided in the starter code to establish a consistent reference point. This baseline establishes the starting CER before introducing any architecture or optimization changes.

2.4 Branch A: architecture and optimization study

This branch examines whether recurrent temporal modeling is more suitable than the provided convolutional temporal encoder for the single-user EMG typing task.

A.1 Recurrent encoder variants. In our first set of modifications, we keep the front-end feature processing pipeline unchanged and replace the original temporal encoder with recurrent alternatives. We evaluate both GRU-based and bidirectional LSTM-based encoders to test whether explicit recurrent sequence modeling improves alignment and character decoding relative to the provided TDSConv baseline. Our recurrent branch is motivated in part by the strong sequence-modeling behavior of gated recurrent units (GRUs) in prior work (4).

A.2 CNN+RNN hybrid encoder variants. We also evaluate hybrid architectures that combine convolutional temporal processing with a recurrent stage. In particular, we test both CNN+GRU and CNN+LSTM variants. These hybrids are designed to determine whether local convolutional feature extraction and recurrent sequence modeling are complementary in this setting.

A.3 Optimization and regularization changes. Beyond architecture alone, we study the effect of optimizer choice and augmentation strength. In particular, we compare Adam vs. AdamW and also compare lighter vs. stronger masking/regularization settings. These experiments help determine whether performance gains come from architecture choice, training dynamics, or both.

A.4 Negative trials. We explicitly retain selected unsuccessful experiments, such as overly strong regularization or larger-capacity settings that degrade performance. Including these failures is important because they clarify which intuitions do *not* hold in this task.

2.5 Branch B: Long-Range Temporal Modeling with Dilated Convolutions

This branch examines whether a purely convolutional temporal encoder with a larger effective receptive field is better suited than the provided baseline for the single-user EMG typing task.

B.1 Dilated residual temporal encoder. In our main modification, we keep the front-end feature processing pipeline unchanged and replace the baseline temporal encoder with a dilated residual temporal convolutional network inspired by temporal convolutional networks for sequence modeling (3). This tests whether exponentially increasing dilation can capture long-range dependencies while preserving stable full-session inference at test time.

B.2 Optimization and augmentation changes. Beyond architecture alone, we also study whether a stronger optimization setup helps this temporal-convolution branch. In particular, our best model combines AdamW with light regularization and lighter SpecAugment settings. These choices are intended to improve optimization stability without overly corrupting the already limited single-user training data.

B.3 Negative trials. We explicitly retain selected unsuccessful experiments, most notably a lightweight Transformer temporal encoder that achieved moderate validation performance but failed under the full-session test protocol. Including this failure is important because it clarifies that long-range modeling alone is not sufficient; the architecture must also remain computationally compatible with the actual evaluation setup.

Our final Branch B model is a DILATED RESIDUAL TCN trained with the same log-spectrogram front end used in the shared pipeline. We retain the multi-band rotation-invariant MLP feature extractor and replace the baseline temporal encoder with a stack of eight residual temporal blocks using exponentially increasing dilation factors. Each block applies a dilated 1D convolution, GELU activation, dropout, and a residual connection with layer normalization. In our best configuration, the front end uses `mlp_features=[384]`, the temporal encoder uses `num_blocks=8` with kernel width 5 and dropout 0.25, and optimization uses AdamW with learning rate 10^{-3} and weight decay 10^{-5}. We also use a lighter SpecAugment configuration with one time mask and one frequency mask.

2.6 Branch C: Bilateral Fusion Architectures

This branch examines whether explicitly modeling the bilateral structure of the dual-wrist EMG input is more effective than using a fixed shared representation. We keep the log-spectrogram front end unchanged and instead vary when and how the left- and right-wrist streams interact inside the encoder.

C.1 Early-fusion temporal encoder. In our first modification, we combine the left- and right-wrist channels immediately after front-end preprocessing and pass the fused representation into a single TDSConv temporal encoder. This tests whether learning cross-wrist interactions from the earliest stage is beneficial.

C.2 Gradual fusion with shared temporal modeling. In our second modification, we first extract compact wrist-specific features and then fuse them before temporal encoding. This creates a more moderate bilateral design that preserves some wrist-specific structure before forming a shared representation.

C.3 Dual-stream late-fusion encoder. In our main Branch C model, we keep the two wrist streams separate through most of the encoding pipeline, apply parallel temporal encoders to each stream, and fuse them only at a later stage before the CTC prediction head. This design is intended to preserve wrist-specific temporal patterns for longer before cross-wrist integration.

Table 1: Best quantitative result from each experiment branch.

Branch	Best model / setting	Key idea	Val CER ↓	Test CER ↓
Shared	TDSConv baseline	Baseline model	18.48	21.85
A	CNN+GRU hybrid	Convolutional front-end with recurrent temporal modeling + shared beam decoding	10.04	9.44
A	CNN+LSTM hybrid	Convolutional front-end with LSTM temporal modeling	14.07	14.76
A	BiLSTM	Bidirectional LSTM temporal encoder	14.86	15.82
B	Dilated Residual TCN	Dilated temporal convolutions + shared beam decoding	17.77	19.47
C	Dual-stream late fusion	Left/right fusion under the shared decoding setup	12.36	11.59

C.4 Optimization and decoding changes. We also evaluate the strongest bilateral architecture with AdamW under the same CTC beam decoding setup as Branch A to determine whether performance gains come from the fusion architecture alone or also from improved optimization.

C.5 Negative trials. We retain weaker bilateral designs, especially strict early fusion, as informative negative results. These experiments help show that bilateral information is useful but that overly early interaction between the two wrist streams can degrade performance.

2.7 Experimental protocol

To make cross-branch comparisons fair, we standardize the reporting format across all experiments:

- each branch states the exact change relative to baseline,
- each experiment reports validation and test CER when available,
- each branch highlights both its best result and at least one informative negative result,
- all final comparisons are collected into a single summary table.

3 Results

3.1 Main quantitative comparison

Table 1 summarizes the best result from each experiment branch together with the shared baseline.

3.2 Branch A results: architecture and optimization

Replacing the baseline temporal encoder with a recurrent architecture substantially improves performance, indicating that explicit sequence modeling is highly effective in this single-user setting. Among the variants we tested, the CNN+GRU hybrid achieved the best result in Branch A, suggesting that a convolutional temporal front-end and recurrent sequence modeling are complementary for this task. We also evaluated LSTM-based alternatives to test whether the gains generalize across recurrent cell types. The CNN+LSTM hybrid achieved a validation/test CER of 14.07/14.76, while the BiLSTM achieved 14.86/15.82. Although both LSTM-based models still support the broader conclusion that recurrent temporal modeling is useful, they underperformed the CNN+GRU model in our current setup. Interestingly, scaling model capacity and masking strength too aggressively leads to a strong performance drop, showing that this task is sensitive to over-regularization. Figure 1 shows the training dynamics of a representative CNN+GRU run used in the masking ablation under CTC greedy decoding. The model trains stably, with rapid CER reduction in the early stage and slower gains later in optimization.

3.3 Branch B results: Long-Range Temporal Modeling with Dilated Convolutions

Replacing the baseline temporal encoder with a dilated residual TCN improves performance, indicating that broader temporal context is useful even without switching to recurrent modeling. Compared

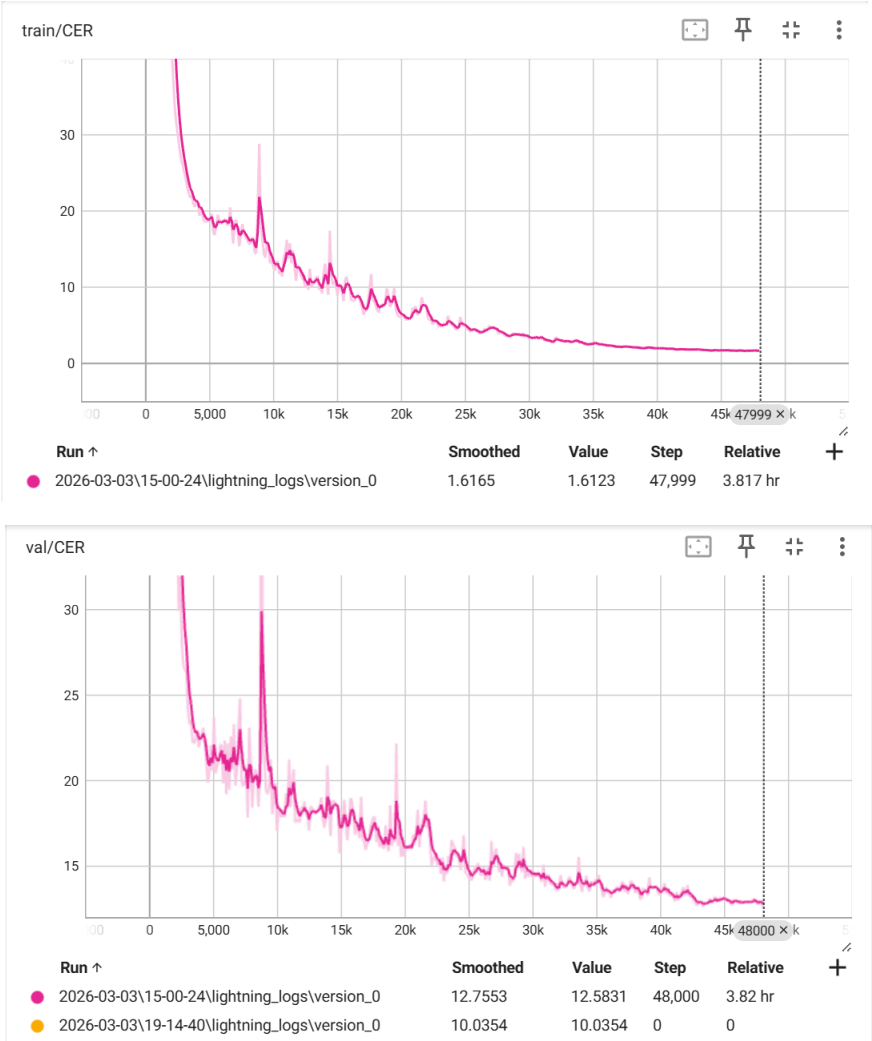

Figure 1: Training dynamics of a representative CNN+GRU run used in the masking ablation under CTC greedy decoding. Top: training-set CER. Bottom: validation-set CER. The model shows stable optimization, with rapid early CER reduction followed by slower improvements later in training. The main results table reports the final branch-best result after shared beam decoding, so its numbers need not match the greedy-decoding curve exactly.

with our reproduced TDSConv baseline, the dilated residual design yields a clear improvement and suggests that the baseline temporal stack does not fully exploit the amount of usable long-range context available in the single-user setting.

An important observation is that architecture practicality matters alongside raw modeling capacity. Our lightweight Transformer temporal encoder appeared promising during validation, but it did not produce a valid final test result because full-session evaluation exposed sequence-length limitations in the model design. In contrast, the dilated TCN preserves long-range temporal modeling while remaining computationally stable under the actual test protocol.

More broadly, this branch suggests that long-context temporal modeling can help substantially, but only when the chosen architecture respects the compute and inference constraints of the benchmark. In our experiments, dilated convolutions provide a better tradeoff between temporal coverage and deployment feasibility than the more expensive full-attention alternative.

Table 2: Ablation on masking strength for the best CNN+GRU model

Mask setting	Val CER ↓	Test CER ↓
No mask	14.86	20.24
Default mask	12.58	14.18
Strong mask	15.04	15.19

3.4 Branch C results: Bilateral Fusion Architectures

Replacing the fixed shared bilateral representation with explicit fusion architectures improves performance, indicating that the interaction between the left- and right-wrist streams is an important design choice in this task. Among the three variants we tested, the strict early-fusion model performed worst, with a validation/test CER of 20.51/23.12, suggesting that forcing the two wrist streams to interact too early can blur useful wrist-specific temporal structure.

A more moderate gradual-fusion design improves over early fusion, achieving 19.14/21.74. This suggests that preserving wrist-specific features for a short stage before merging is more effective than immediate joint encoding, but still does not fully exploit the bilateral structure of the signal.

Our best Branch C model is the dual-stream late-fusion encoder, which keeps the two wrist streams separate through most of temporal modeling and fuses them only at a later stage. Without additional optimization changes, this model reaches 18.85/20.55 under the shared decoding setup, already outperforming the other bilateral variants. With AdamW and CTC beam decoder, its performance improves further to 12.36/11.59, making it the strongest model in Branch C.

More broadly, Branch C suggests that bilateral information is indeed useful, but only when the architecture preserves wrist-specific temporal patterns long enough before fusion. In our setting, late fusion is clearly more effective than either strict early fusion or intermediate shared fusion.

3.5 Ablation on masking strength for the best CNN+GRU model

To better understand the role of augmentation in our best architecture, we performed an ablation on masking strength using the CNN+GRU model as the reference backbone. For consistency, the masking ablation is reported under CTC greedy decoding, while the main summary table reports the final branch-best result after the shared beam decoder is applied. Our default setting applies one time mask and one frequency mask in a SpecAugment-style masking scheme (2). We then compared this setting against a no-mask variant and a stronger-mask variant.

The default masking configuration achieved the best result, with a validation/test CER of 12.58/14.18. Removing masking degraded performance to 14.86/20.24, indicating that some degree of augmentation is important for generalization in this benchmark. Increasing the masking strength also failed to improve over the default setting, yielding 15.04/15.19 in the current run. Taken together, these results suggest that moderate masking is beneficial, whereas stronger masking makes optimization harder and does not automatically improve final performance.

At the same time, we do not view this as conclusive evidence against stronger masking. A heavier augmentation setting may require either a longer training schedule or a larger model before its potential benefit can be fairly assessed. These runs are only partially controlled, since training budget and model capacity are not identical across all masking settings. Our current results therefore point to an optimization tradeoff: masking helps, but overly aggressive masking can reduce performance under the present training setup.

4 Discussion

Our experiments suggest that the largest gains in our current setup are associated with improved temporal modeling rather than with simply increasing model complexity, although some of these improvements may also reflect differences in optimization and decoding. In Branch A, a CNN+GRU hybrid encoder outperforms the provided TDSConv baseline in our runs, suggesting that combining local convolutional processing with recurrent sequence modeling can be effective for this single-user decoding task. Branch B suggests that longer-range temporal context can help even in a purely

convolutional design, while Branch C suggests that preserving left- and right-wrist structure until a later fusion stage can be beneficial.

A second theme is that optimization and regularization must be tuned carefully. Moderate changes can improve performance, but stronger masking or larger-capacity settings do not automatically help and may even hurt. This suggests that the benchmark is sensitive to over-regularization and that stable training remains important.

A follow-up masking study further shows that augmentation is important in this benchmark: removing masking clearly hurts performance, while overly strong masking also underperforms the default setting, suggesting a tradeoff between regularization strength and optimization difficulty.

Limitations. This report focuses on the provided single-user benchmark under limited time and compute. Some conclusions may change under broader hyperparameter sweeps, multi-user settings, or more extensive decoding experiments.

Final takeaway. Final takeaway. Taken together, the results indicate that temporal modeling, fusion design, and training choices all affect in this benchmark. In our experiments, the strongest results are obtained by models that capture temporal structure well while remaining trainable and stable under the shared decoding setup.

References

[1] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 369–376, 2006.

[2] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. *arXiv preprint arXiv:1904.08779*, 2019.

[3] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. *arXiv preprint arXiv:1803.01271*, 2018.

[4] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. In *Proceedings of EMNLP*, pages 1724–1734, 2014.

[5] Awni Hannun, Ann Lee, Qiantong Xu, and Ronan Collobert. Sequence-to-Sequence Speech Recognition with Time-Depth Separable Convolutions. *arXiv preprint arXiv:1904.02619*, 2019.

[6] Viswanath Sivakumar et al. emg2qwerty: A Large Dataset with Baselines for Touch Typing using Surface Electromyography. *arXiv preprint arXiv:2410.20081*, 2024.